

Task 1

```
package Day25;
public abstract class Shape { 3 usages 1 inheritor
    protected ExcalidrawAPI excalidrawAPI; 2 usages

    protected Shape(ExcalidrawAPI excalidrawAPI) { this.excalidrawAPI = excalidrawAPI; }

    public abstract void draw(); 2 usages 1 implementation
}
```

```
package Day25;
public class Square extends Shape { 2 usages
    private int s; 2 usages
    public Square(int s, ExcalidrawAPI excalidrawAPI) { 2 usages
        super(excalidrawAPI);
        this.s = s;
    }
    @Override 2 usages
    public void draw() {
        excalidrawAPI.drawSquare(s);
    }
}
```

```
package Day25;
public class DrawingFrame implements ExcalidrawAPI { 1 usage
    @Override 1 usage
    public void drawSquare(int s) {
        System.out.println("draw square in Drawing frame using Excalidraw with side " + s);
    }
}
```

```
package Day25;
public class DrawingPicture implements ExcalidrawAPI { 1 usage
    @Override 1 usage
    public void drawSquare(int s) {
        System.out.println("draw square in Drawing Picture using Excalidraw with side " + s);
    }
}
```

```
package Day25;

public interface ExcalidrawAPI { 7 usages 2 implementations
    void drawSquare(int s); 1 usage 2 implementations
}
```

```
package Day25;
public class BP {
    public static void main(String[] args) {
        System.out.println("Bridge Method Design Pattern");

        ExcalidrawAPI obj1 = new DrawingFrame();
        ExcalidrawAPI obj2 = new DrawingPicture();

        Shape square1 = new Square(s: 5, obj1);
        square1.draw();

        Shape square2 = new Square(s: 7, obj2);
        square2.draw();
    }
}
```

```
"D:\Program Files\Java\jdk-24\bin\java.exe" "-jav
Bridge Method Design Pattern - Structural DP!
draw square in Drawing frame using Excalidraw wit
draw square in Drawing Picture using Excalidraw w

Process finished with exit code 0
```

Task2

```
package Day25;

public interface Company { no usages 2 implementations
    void displayName(); no usages 2 implementations
}
```

```
package Day25;
import java.util.ArrayList;
import java.util.List;
// Composite component
public class CompanyHead implements Company { 2 usages
    private int id; 2 usages
    private String name; 2 usages
    private List<Company> subDepartments; 4 usages
    public CompanyHead(int id, String name) { 1 usage
        this.id = id;
        this.name = name;
        this.subDepartments = new ArrayList<>();
    }
    @Override 2 usages
    public void displayName() {
        System.out.println("Company Head: " + name + " (ID: " + id + ")");
        subDepartments.forEach(Company::displayName);
    }
    public void addDepartment(Company company) { 1 usage
        subDepartments.add(company);
    }
    public void removeDepartment(Company company) { no usages
        subDepartments.remove(company);
    }
}
```

```

package Day25;
// Leaf component
public class Software implements Company { 1 usage
    private int id; 4 usages
    private String name; 4 usages

    public Software(int id, String name) { 1 usage
        this.id = id;
        this.name = name;
    }
    @Override 2 usages
    public void displayName() {
        System.out.println("    Software Department: " + name + " (ID: "
    }
    // Getters
    public int getId() { no usages
        return id;
    }
    public String getName() {
        return name;
    }
    // Setters
    public void setId(int id) { no usages
        this.id = id;
    }

```

```

package Day25;
public class CompositeMethodDP {
    public static void main(String[] args) {
        System.out.println("Composite Method DP - Structural DP");
        // Leaf
        Company softwareDept = new Software(id: 1, name: "Application Department");
        // Composite
        CompanyHead companyHead = new CompanyHead(id: 3, name: "ABC Company");
        companyHead.addDepartment(softwareDept);
        // Display structure
        companyHead.displayName();
    }
}

```

"D:\Program Files\Java\jdk-24\bin\java
 Composite Method DP - Structural DP
 Company Head: ABC Company (ID: 3)
 Software Department: Application De
 Process finished with exit code 0

Task 3

```
package Day25;
public class BillingCounterService { 2 usages
    public void payBill(String accountId, String billId, double amount) { 1 usage
        System.out.println(
            "Paying " + amount +
            " for billId " + billId +
            " from account " + accountId
        );
    }
}
```

```
package Day25;
public class MallFacade { 2 usages
    private StoreStaffService storeStaffService; 2 usages
    private BillingCounterService billingCounterService; 2 usages

    public MallFacade() { 1 usage
        this.storeStaffService = new StoreStaffService();
        this.billingCounterService = new BillingCounterService();
    }

    public void getItems(String items) { storeStaffService.getItems(items); }

    public void payBill(String accountId, String billId, double amount) { 1 usage
        billingCounterService.payBill(accountId, billId, amount);
    }
}
```

```
package Day25;
public class StoreStaffService { 2 usages
    public void getItems(String items) { System.out.println("Selecting " + items +
}
```

```
package Day25;
public class FacadeMain {
    public static void main(String[] args) {
        System.out.println("Facade Method DP - Structural Design Pattern");

        MallFacade mallFacade = new MallFacade();

        mallFacade.getItems("Fruits");
        mallFacade.getItems("Electronics");

        mallFacade.payBill(accountId: "123", billId: "billno 44");
    }
}
```

```
"D:\Program Files\Java\jdk-24\bin\java.exe" "-D:
Facade Method DP - Structural Design Pattern
Selecting Fruits from the store...
Selecting Electronics from the store...
Paying 500.0 for billId billno 44 from account
Process finished with exit code 0
```